

算出・判定ロジック仕様（202607）

数値の求め方と自動判定のルールをまとめた仕様。画面から見た機能は [funcspec-202607.md](#)、実装配置は [design-202607.md](#) を参照。

本書は、個別に書かれていた3つの実装仕様書（収束による自動停止 → 3. / 衛星表示のキャリーフォワード → 4. / 受信品質統計 → 5.）を統合したもの。

章	実装
1. 座標とメートルの換算 / 2. 精度と集計 / 3. 収束判定	js/accuracy.js （純粋関数）
3. 収束の適用	js/recorder.js
4. 衛星表示の保持	js/charts.js
5. 受信品質統計	js/stream-stats.js

1. 座標とメートルの換算

局所平面近似で足りる（1 地点のばらつき評価が目的で、扱う距離は数十 m 以下）。

```
latM = 111320 // 南北方向 [m/度]
lonM = 111320 × cos(lat[rad]) // 東西方向 [m/度]
```

中心からのオフセットは [東西](#) $e = (lon - lonMean) \times lonM$ / [南北](#) $n = (lat - latMean) \times latM$ 。

2. 精度の推定と静止点の集計

2-1. 水平精度の推定（1 エポック）

優先順に評価し、最初に成立したものを採用する。

優先	条件	値	表示ソース
1	GST の緯度・経度標準偏差がある	$\sqrt{latStd^2 + lonStd^2}$	GST
2	HDOP がある	$HDOP \times UERE$ （UERE は設定値、既定 5 m）	HDOP×UERE
3	どちらも無い	表示なし（-）	—

GST はレシーバが出力する擬似距離誤差統計に基づく値、HDOP×UERE は概算値である点に注意。

2-2. 記録の集計（全エポック）

対象は「緯度経度がある」エポック。1 件も無ければ集計値は生成しない。

指標	定義
center	緯度・経度の 算術平均
median	緯度・経度の 中央値 （外れ値の影響を見るため併記）
stdEastM / stdNorthM	中心からの東西 / 南北オフセットの標準偏差（不偏、n-1）。平均は 0 とし て計算
drms	$\sqrt{(\text{stdEastM}^2 + \text{stdNorthM}^2)}$... 水平方向の RMS ばらつき
drms2	$\text{drms} \times 2$ (2DRMS)
cep50 / cep95	中心からの半径誤差を昇順に並べた 経験的パーセンタイル （50% / 95%、 線形補間）
altMean / altStd	標高(MSL)の平均・標準偏差（値のあるエポックのみ）
count	集計に使ったエポック数
fixCounts	fix quality ごとの件数
avgPdop / avgHdop / avgVdop	各 DOP の平均（値のあるエポックのみ）
avgSats	使用衛星数の平均
avgSnrUsed	使用中 の衛星の C/N0 の平均（可視のみの衛星は除く）
offsets	各エポックの {e, n}（散布図用）

- CEP は理論式ではなく**実測分布のパーセンタイル**。したがって点数が少ないと不安定。
- これらは「点のばらつき（精密さ）」であり、真位置からの誤差（正確さ）ではない。

3. 収束による自動停止

3-1. 前提

- 単周波・自律測位の受信機のため、**「収束」＝「これ以上データを足しても推定が動かない（頭打ち）」**であって、**精度の保証ではない**。マルチパスで“安定して偏った”値に張り付くことがあるため、**品質ゲートと上限時間を必ず併用する**。
- 中心と DRMS は累積統計なので時間とともに必ず平坦化する。したがって収束判定は **直近 holdSec 秒の窓**での変動で見る（累積値の単純な差分では早期停止してしまう）。
- 偽収束（マルチパスで偏った値に張り付いたまま安定する）は **minSec** を延ばせば抑えられるが、そのコストは見通しの良い地点にも等しく乗る（開けた場所での所要時間は実質 **minSec** で決まる）。本アプリでは **minSec は 30 秒に据えて偽収束をある程度許容し**、その代わりに Android内蔵GNSS との同時取得（[funcspec-202607.md](#) 4-8）と、調査日の対応表CSV による 事後検証（DRMS 比・中心のズレ・平均HDOP）で疑わしい地点を拾う方針とする。

3-2. 品質ゲート

そのエポックを収束判定に使えるかの条件。

```
fixMode === 3 // 3D 測位
&& hdop != null && hdop <= 3
&& satsUsed != null && satsUsed >= 5
```

- 満たさないエポックが来たら**収束用の履歴をリセット**する（「安定してから 10 秒」を数え直す）。
- **記録データは間引かない**。蓄積条件は `lat != null && lon != null && fixQuality > 0` のままで、品質ゲートは停止判定にのみ用いる。

3-3. 判定（毎エポック = 1 Hz）

品質ゲートを通ったエポックについて { `t`: 経過秒, `lat`, `lon`, `drms` }（いずれも**その時点の集計値**）を履歴に積み、次を評価する。

1. `elapsedSec < minSec` または履歴が 2 件未満 → 判定しない (`stable: false`)。
2. `cutoff = elapsedSec - holdSec` 以前で最も新しい履歴を**基準点 ref** とする。存在しなければ窓を満たしていない (`stable: false`、表示は「判定中」)。
3. `centerMoveM` = 基準点と最新の中心間の距離 [m] (1. の換算を使用)。
4. `drmsRangeM` = 窓内 (`t ≥ ref.t`) の DRMS の**最大 - 最小**。
5. 判定：

```
drmsTol = max(drmsTolAbsM, drmsTolPct × 現在のDRMS)
stable = (centerMoveM <= centerTolM) && (drmsRangeM <= drmsTol)
```

履歴は基準点を確保できる範囲で間引く (`holdSec + 5` 秒より古いものを捨てる)。

3-3-1. 戻り値と進捗表示（2 フェーズ）

判定は **2 段階** になっている。記録タブの進捗バーもこの 2 段で表示する（[funcspec-202607.md](#) 4-4）。1 本のバーで表すと、窓が揃ってバーが満タンになっても中心が動いていれば停止しない状態を説明できないため。

フェーズ	状態	バーの表示
1	品質ゲートを通るエポックが連続 <code>holdSec</code> 秒たまるまで	「安定待ち」 = <code>stableSec / holdSec</code> 。品質不良で履歴がリセットされると 0 に戻る（色を変えて戻ったことを示す）
2	窓は揃った。中心移動・DRMS 幅が許容内へ落ち着くのを待つ	「判定中」 = 中心移動・DRMS 幅の「実測値 / 許容値」をゲージで表示

`evaluateConvergence` の戻り値:

項目	内容
<code>stable</code>	収束成立（＝自動停止する）

項目	内容
<code>stableSec</code>	連続した良好データの長さ [秒]。 <code>elapsedSec</code> - 履歴の先頭 <code>.t</code> 。履歴を間引くため <code>holdSec</code> + 5 秒で頭打ちになる
<code>windowReady</code>	基準点が取れた（フェーズ2に入った）か
<code>centerMoveM</code> / <code>drmsRangeM</code>	3-3. の実測値。フェーズ1では <code>null</code>
<code>centerTolM</code> / <code>drmsTolM</code>	対応する許容値。バーの達成度を描くために返す

3-4. パラメータ

名前	既定値	種別	説明
<code>autoStop</code>	<code>true</code>	設定 (<code>js/constants.js</code>)	収束による自動停止 ON/OFF
<code>minSec</code>	30 秒	設定 (<code>js/constants.js</code>)	これ未満では絶対に停止しない
<code>maxSec</code>	180 秒	設定 (<code>js/constants.js</code>)	上限＝タイムアウト。0 で無制限
<code>maxEpochs</code>	300	設定 (<code>js/constants.js</code>)	上限エポック数。0 で無制限
<code>holdSec</code>	10 秒	定数	収束が維持されるべき時間（判定窓）
<code>centerTolM</code>	0.3 m	定数	窓内の中心の許容移動量
<code>drmsTolAbsM</code>	0.3 m	定数	DRMS の許容変動（絶対）
<code>drmsTolPct</code>	0.05（5%）	定数	DRMS の許容変動（相対）
<code>beep</code>	<code>true</code>	設定 (<code>js/constants.js</code>)	自動停止をビープ音で知らせる（3-5）
<code>STALL_MS</code>	10 秒	定数 (<code>js/recorder.js</code>)	最後のエポックからこの時間で途絶停止
<code>FLUSH_EPOCHS</code>	5	定数 (<code>js/recorder.js</code>)	記録中の追記の粒度（何エポックを1件にまとめるか）
<code>MAX_FLUSH_FAILURES</code>	3	定数 (<code>js/recorder.js</code>)	追記がこの回数続けて失敗したら打ち切る
<code>INSUFFICIENT</code>	30 秒 / 10 点	定数 (<code>js/recorder.js</code>)	この時間を超えて点数が足りなければ破棄を確認する

定数は画面の煩雑化を避けるためモジュール定数（`js/recorder.js` の `CONVERGENCE`）とし、設定画面には出さない。一方「設定」種別の既定値は `js/constants.js` の `DEFAULT_SETTINGS` にまとめてあり、設定タブから起動中のみ変更できる（永続化しない → `funcspec-202607.md` 7.）。

3-5. 停止理由と知らせ方

停止時に `stopReason` と `autoStop`（収束判定を働かせていたか）を記録に残し、測位結果の見出しへ 1 行で出す（`js/view-utils.js` の `stopSummaryText`）。

値	契機	ビープ音	見出し
<code>converged</code>	3-3. の <code>stable</code> が成立	短く 2 回（2400 Hz / 0.12 秒×2）	収束で自動停止（45秒/45点）
<code>timeout</code>	<code>maxSec > 0 && elapsedSec >= maxSec</code>	長く 1 回（900 Hz / 0.6 秒）	上限時間で停止（未収束：180秒/180点）
<code>maxEpochs</code>	<code>maxEpochs > 0 && 点数 >= maxEpochs</code>	長く 1 回（900 Hz / 0.6 秒）	上限エポックで停止（未収束：150秒/300点）
<code>manual</code>	停止ボタン	鳴らさない	手動停止（未収束：20秒/20点）
<code>stalled</code>	最後のエポックから 10 秒（ <code>STALL_MS</code> ）	長く 1 回（900 Hz / 0.6 秒）	データ途絶で停止（未収束：45秒/40点）
<code>storageError</code>	記録中の追記が連続 3 回失敗	長く 1 回（900 Hz / 0.6 秒）	保存失敗で停止（未収束：60秒/58点）
<code>crashed</code>	停止処理が走らないまま落ちた 下書きを確定した	—	記録中に中断（未収束：40秒/38点）

`stalled` は 1 秒間隔のタイマーで判定する。上の 3 つ（`converged` / `timeout` / `maxEpochs`）はエポック到着時にしか評価されないため、データが途絶えると記録が止まらない。これを塞ぐのが `stalled` の役目で、画面 OFF・BLE 切断・受信機の電池切れ・fix 喪失を一様に拾う。基準にするのは「蓄積対象になったエポックを受けた時刻」なので、**fix 喪失も途絶として扱う**。

旧版の `interrupted`（画面 OFF が猶予 5 秒を超えた）は発生しなくなったが、既存データを読めるよう表示ラベルだけ残している。

見出しの組み立て（`stopSummaryText`）：

- 点数が 0 なら停止理由によらず **測位データなしで停止（30秒/0点）**。
- 但し書きは `converged` のみ無し。それ以外は `autoStop === false` なら **収束判定なし**：、そうでなければ **未収束**。自動停止 OFF のときは「未収束」ではない（判定していないため）。`autoStop` を持たない旧データは「未収束」に寄せる。
- 括弧内は `${秒数}秒/${点数}点`。1 Hz なら本来一致するはずの 2 つがずれていれば、取りこぼし（BLE 欠落・中断）があったと読める。
- 停止理由は**この見出しだけ**が持つ。集計テキスト（`formatStats`）には含めない（同じ内容を 2 か所に出さない）。

音を鳴らす条件（`js/beep.js`）：

- 屋外のスマホスピーカーは低音がほとんど出ないため、高めの周波数を使う。
- `AudioContext` は自動再生ポリシーにより**ユーザー操作のハンドラ内でしか作れない**。記録ボタンのクリック中に用意する（停止時に初めて作っても鳴らない）。

- 手動停止で鳴らさないのは、操作した本人が停止を分かっているため。データ途絶・保存失敗は本人が停止を分かっていないので鳴らす。
- 端末が消音・音量 0 なら鳴らない（それでよい。振動は使わない）。

4. 衛星表示のキャリーフォワード

4-1. 課題

スカイプロットと SNR チャートは毎回クリアしてから描くため、GSV が届かない秒には表示が空になり点滅して見える（GSV は帯域や欠落の影響で每秒そろわないことがある）。

4-2. 判定（純粋関数 `holdDecision`）

```
holdDecision(hasSats, ageMs, holdMs):
    hasSats          → 'draw'      // 描画可能な衛星データがある：通常描画
    ageMs <= holdMs  → 'hold'      // 失効時間内：前フレームを保持（クリアも描画もしない）
    それ以外          → 'clear'    // 失効：枠（グリッド/目盛り）だけ描いてクリア
```

記号	意味
<code>hasSats</code>	スカイプロットは仰角・方位を持つ衛星、SNR は C/N0 を持つ衛星が 1 つ以上あるか
<code>ageMs</code>	最後に有効データを描画してからの経過（未受信は ∞ → 初期状態は <code>clear</code> ）
<code>holdMs</code>	保持上限 <code>HOLD_MS = 8000</code> （両ビュー共通の定数）

- 経過時間の基準はエポックの受信時刻（`recvAt`）。読込データを表示する場合は表示時刻を `recvAt` として補うため、ライブと同じ経路で判定される。
- 保持中のリサイズ・タブ再表示では、**最後の有効エポック**で再描画する（`redraw()`）。
- 系統別チップも、内訳が空のときは前の表示を保持する。

5. 受信品質統計

5-1. 考え方

GNSS受信機 --UART--> Pico --BLE(NUS)--> アプリ の各段で「入った数」と「出た数」を数え、区間差分を突き合わせれば、どの段で・どれだけ取りこぼしたかが分かる。

```
GNSS受信機 —UART—> Pico —BLE notify—> LineBuffer —> parser —>
EpochAssembler
欠落モード： ①UART溢れ/化け ②BLE送信破棄 ③通知消失 ④CS NG ⑤未対応文 ⑥工
ポック/GSV欠け
検出手段： Pico側 ng/drop Pico側 txng Δtxok-Δ受信行数 csNg計数 unknown計数
時刻ギャップ/msgNum
```

Pico 側のカウンタ \$PPICO の仕様は `hardware-202607.md` 4. を参照。

5-2. アプリ側のカウンタ

`reset()`（接続開始・モック開始・手動リセット）起点の累計。1 秒ごとに接続タブへ描画する。

カウンタ	供給元	意味
<code>lines</code>	受信フレーム	LineBuffer が復元した全行（ <code>\$PPICO</code> 含む）
<code>csNg</code>	パーサ	チェックサム不一致（ 計数して破棄 ）
<code>parsedOk</code>	パーサ	内容まで解釈した行（GGA/RMC/GSA/GSV/VTG/GST）
<code>unknown</code>	パーサ	チェックサム正常だが未対応の種別（ 計数のみで無視 ）
<code>discardedChars</code>	LineBuffer	行にならないまま捨てた文字数（4096 字超で破棄）
<code>epochs / epochGaps</code>	エポック確定	確定エポック数 / 時刻ギャップ（1.5 秒超）から推定した欠落エポック数
<code>gsvMissing</code>	エポック確定	GSV グループ（talker × signalId）で届かなかった msgNum の数
<code>pico.* / picoSeqGaps</code>	<code>\$PPICO</code>	Pico 側カウンタの最新値 / <code>seq</code> 抜けの数
<code>bleLossEst</code>	突合	BLE 通知欠落の推定（下記）

5-3. BLE 欠落の推定

最初の `$PPICO` 受信時点の `txok` とアプリの受信行数を基準点として対応付け、以降は

$$\text{bleLossEst} = (\text{txok} - \text{txok基準}) - (\text{受信行数} - \text{受信行数基準}) \quad \text{※負は 0 に丸める}$$

- 化けて届いた行も「受信した行」として数える（`csNg` 行き）ため、これは**通知ごと消えた分の推定**。
- BLE の in-flight データの分だけ基準点に数行のずれが乗り得るので、±数行は誤差とみなす。
- `txok` または `seq` が後退したら Pico 再起動とみなし、基準点を取り直す（推定値は 0 に戻る）。
- `$PPICO` を 1 度も受けていなければ `null`（表示は -）。

5-4. 取りこぼし時の対応

事象	対応
チェックサム NG 行	<code>csNg</code> に計数して破棄。接続・処理は継続
未対応センテンス	<code>unknown</code> に計数のみ
行にならないゴミ	LineBuffer が 4096 字超で破棄し、文字数を <code>discardedChars</code> に計上
<code>\$PPICO</code>	統計にのみ反映し、エポック組み立てには回さない

事象	対応
描画が追いつかない	<code>requestAnimationFrame</code> で最新エポックのみ描画（データ処理自体は間引かない）

BLE notify はブラウザのイベントキューに積まれるため、アプリの処理が一時的に遅れても データは失われない（失われるのは BLE リンク層で、それは `bleLossEst` に現れる）。

5-5. 記録への保存（区間差分）

記録 1 回分の区間差分（停止時の値 - 開始時の値）を `summary.rxStats` に保存し、結果テキストの末尾に表示する。区間開始時に `$PPICO` を未受信だった場合、Pico カウンタの差分は帰属できないため `pico:null`（表示は「Pico統計（\$PPICO）なし」）とする。

5-6. 注意

- 接続直後の初回エポックは途中受信のため GSV 部分欠落に 1 回分計上され得る（誤差として無視してよい）。
- モック配信も `$PPICO` を模擬する（欠落ゼロの整合値 → BLE 欠落推定は 0 になるはず）。
- Pico のカウンタは起動からの累計で、切断・再接続ではリセットされない（アプリ側が差分を取る）。
- **すべて 0（BLE 欠落 0・Pico NG 0）なら「受信機 → Pico → アプリまで取りこぼしなし」と確認できる。**

6. 散布図の表示半径

散布図（`funcspec-202607.md` 5-4）で、どこまでを画面内に収めるかの決め方。

6-1. 課題

2 系統を重ねると、Android内蔵のばらつきは GNSS受信機の数倍～10 倍になるのが普通で、さらに Fused Location は中心そのものが数十 m ずれることもある。**点の最大半径**でスケールを決めると、内蔵の外れ点 1 つで受信機の点群が中心の 数ピクセルへ潰れ、肝心のばらつきが読めなくなる。

6-2. 算出

```
target = max(2 m, GNSS受信機のDRMS × 4, Android内蔵のDRMS × 2)
R      = niceCeil(target)                // 段階へ切り上げ
```

- **基準は DRMS**（外れ値に強い）。最大半径や CEP95 だと外れ点 1 つでスケールが吹き飛ぶ。
- 受信機を ×4 にするのは、`2DRMS`（≒95%）の 2 倍にあたり点群が半径の約半分を占めるため。内蔵は ×2 に留める（×4 にすると受信機側が潰れる）。
- 下限 2 m は、記録開始直後（DRMS ≒ 0）にスケールが極端に小さくなるのを防ぐため。

6-3. 段階とヒステリシス

```
RADIUS_STEPS = [1, 1.5, 2, 3, 4, 5, 6, 8] × 10^n
```

- 目盛りの 1/2/5 より**細かく刻む**。粗いと DRMS が少し増えただけで半径が倍になり（例: $4 \times 1.27 = 5.08 \text{ m} \rightarrow 10 \text{ m}$ ）、点群が中心の小さな塊に見えてしまう。この刻みなら行き過ぎは最大 1.5 倍に収まり、点群は常に半径の 30～50% を占める。
- **拡大は即座に、縮小は `target` が現在の半径の半分を下回るまで待つ**（ヒステリシス）。記録中は毎秒 DRMS が動くため、これがないとスケールがばたついて点が動き回る。
- 別の記録へ切り替えるときはヒステリシスを解除する（`resetScale()`）。記録開始時・読込データの切替時に呼ぶ。

6-4. 表示範囲の外に出た点

- 半径 R を超えた点は**縁にクランプして外向きの▲**で描き、系統別に件数を数える。件数だけでなく方向も残るので、どちらへ飛んだかが読める。
- 件数は凡例の下にテキストで出す（**表示範囲の外側に GNSS受信機 2点 / Android内蔵 5点（▲は方向）**）。
- 内蔵の**中心そのもの**が R の外にある場合は、内蔵の中心マーカと DRMS 円を描かない（点は▲として縁に出る）。中心のズレの数値は集計テキストで読む。
- 目盛りリングは R より外には描かない。R は範囲外判定の境界なので、その外にリングがあると「そこまで表示されている」と誤解される。

7. テストで担保している項目

`node tools/check-syntax.mjs`（純粋関数の単体テスト）で次を検証している。

対象	検証内容
チェックサム	正しい文の受理・不正な文の棄却
パース	GGA の座標変換・フィールド、GST、VTG、 <code>\$PPICO</code> 、GSV の <code>signalId</code>
行復元	チャンク断片の結合、溢れ破棄の通知
精度推定	GST 優先、HDOP×UERE フォールバック
集計	点数・平均緯度・DRMS/CEP の大小関係・散布図オフセット・使用衛星の平均 C/N0・m/度換算
収束判定	横ばい列で成立／ドリフト列・DRMS 変動列・時間不足・窓未充足で不成立
収束の進捗	フェーズ1/2 の切替（ <code>windowReady</code> ）・ <code>stableSec</code> ・許容値の返却・DRMS 許容の絶対/相対
停止サマリ	停止理由ごとの文言・「未収束」と「収束判定なし」の書き分け・点数0・旧データ
表示保持	<code>holdDecision</code> の 4 パターン
受信品質	行分類・BLE 欠落推定・ <code>\$PPICO</code> 欠落検出・Pico 再起動・エポックギャップ・区間差分
記録フロー	開始時の下書き作成・5 エポックごとの追記・停止で集計値を書く・確定で採番
下書き	番号を消費しない・削除でチャンクも消える・0 点は自動削除・追記失敗で打ち切る
データ不足	30 秒 / 10 点の判定（0 点・30 秒未満は別扱い）

対象	検証内容
既定地点名	同日連番の採番規則